3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

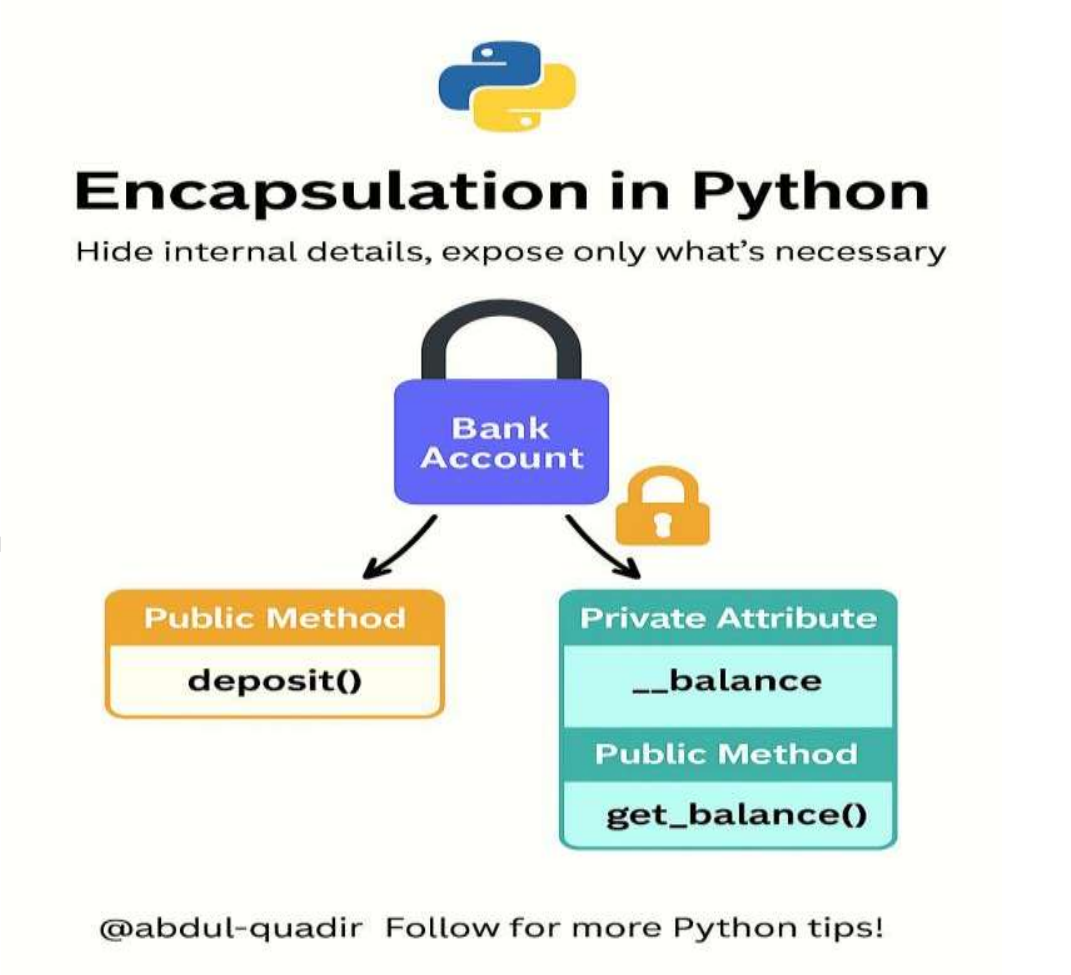
Classes : Offline & Online

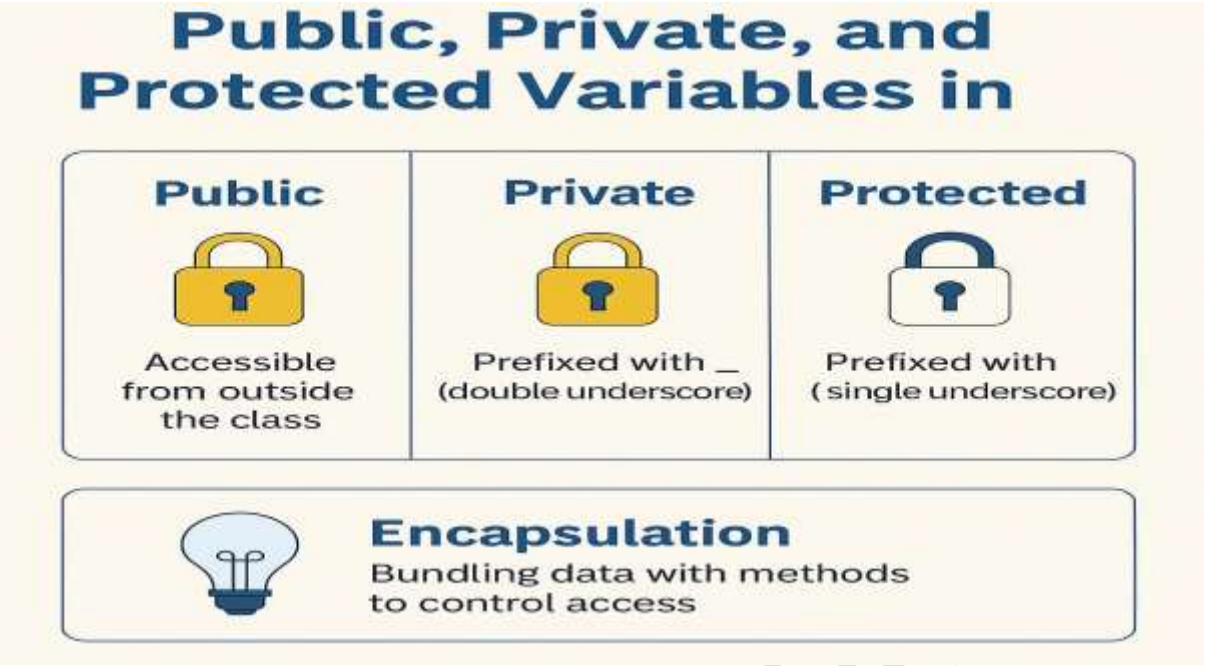
Daily Teaching : 2 Hours

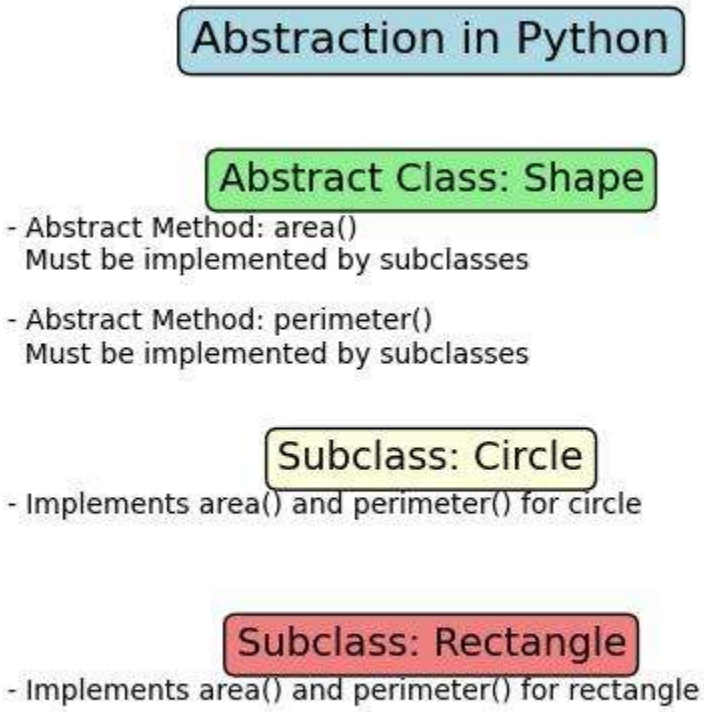
Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 22 – Encapsulation & Abstraction







1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand **Encapsulation** in Python.
- Understand **Abstraction** in Python.
- Explain the difference between encapsulation and abstraction.
- Understand public, protected, and private members.
- Use _ and __ naming conventions.
- Create controlled access using methods.
- Understand getters and setters.
- Use Python properties.
- Create abstract classes using ABC.
- Use @abstractmethod.
- Build a **secure Bank Account class**.
- Apply secure class design to real-world applications.

3DWEBSOFT FOUNDATION

2. What is Encapsulation?

Encapsulation means combining data and the methods that operate on that data inside a class and controlling how that data is accessed.

In simple words:

Encapsulation protects an object's data from inappropriate direct access.

For example, consider a bank account.

We should not allow anyone to directly change:

Balance = ₹1,00,000

to:

Balance = ₹10,00,000

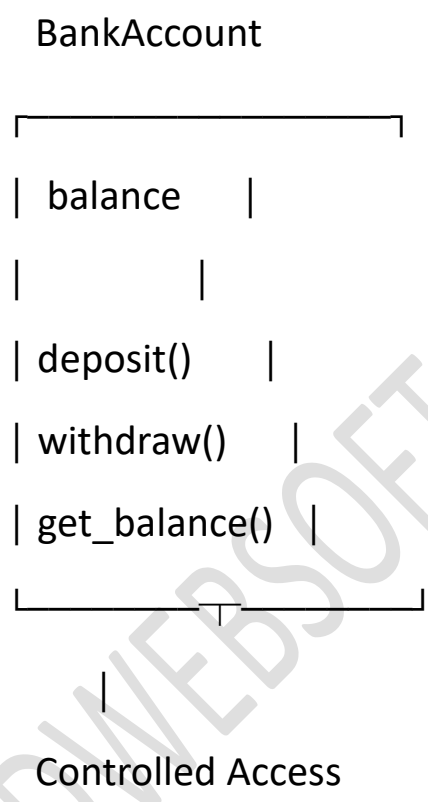
Instead, we provide controlled methods such as:

deposit()

withdraw()

get_balance()

Visual idea



3. Why is Encapsulation Important?

Encapsulation helps us:

- Protect data
- Prevent accidental changes
- Validate input

3DWEBSOFT FOUNDATION

- Control access
- Improve security
- Make programs easier to maintain

Example

Without protection:

account.balance = -50000

This could create an invalid account balance.

With controlled access, we can check:

```
if amount > 0:
    # allow deposit
```

4. Public, Protected and Private Members

Python commonly uses naming conventions to indicate different levels of intended access.

Type	Example	Meaning
Public	name	Can be accessed normally
Protected	_salary	Intended for class/subclass use
Private	__pin	Name-mangled to discourage direct access

5. Public Members

A public attribute can be accessed directly.

class Student:

3DWEBSOFT FOUNDATION

```
    def __init__(self, name):
        self.name = name
s1 = Student("Asha")
print(s1.name)
```

Output

Asha

Here:

self.name

is public.

6. Protected Members

A single underscore is commonly used to indicate a protected/internal attribute.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self._salary = salary
```

We can access it technically:

```
e1 = Employee("Rahul", 50000)
print(e1._salary)
```

But _salary communicates:

"This is intended for internal or subclass use."

3DWEBSOFT FOUNDATION

Important

Python does **not** enforce protected access like some languages do.

7. Private Members

A double underscore is used to create a private-style attribute.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance
```

Now:

```
account = BankAccount(5000)
```

The intended design is that users should not directly access:

```
account.__balance
```

Instead, we provide a method:

```
def get_balance(self):
    return self.__balance
```

8. Encapsulation Example

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance
    def get_balance(self):
        return self.__balance
```

3DWEBSOFT FOUNDATION

```
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        else:
            print("Invalid amount")
account = BankAccount(5000)
account.deposit(2000)
print("Balance:", account.get_balance())
```

Output

Balance: 7000

The balance is controlled through methods.

9. Getter and Setter

Two common terms students should know are:

Getter

A method used to **read/access** data.

```
def get_balance(self):
    return self.__balance
```

Setter

A method used to **modify/update** data.

```
def set_balance(self, amount):
    self.__balance = amount
```

However, setters should normally validate data before changing it.

3DWEBSOFT FOUNDATION

10. Secure Setter Example

```
class Student:


    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks
    def get_marks(self):
        return self.__marks
    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Marks must be between 0 and 100")
s1 = Student("Asha", 85)
print("Marks:", s1.get_marks())
s1.set_marks(92)
print("Updated Marks:", s1.get_marks())
```

Output

Marks: 85

Updated Marks: 92

If we try:

s1.set_marks(150)

Output:

3DWEBSOFT FOUNDATION

Marks must be between 0 and 100

This is **controlled access**.

11. What is Abstraction?

Abstraction means hiding unnecessary implementation details and showing only the important functionality.

In simple words:

Abstraction tells us WHAT an object does, while hiding HOW it does it.

Real-world example: ATM

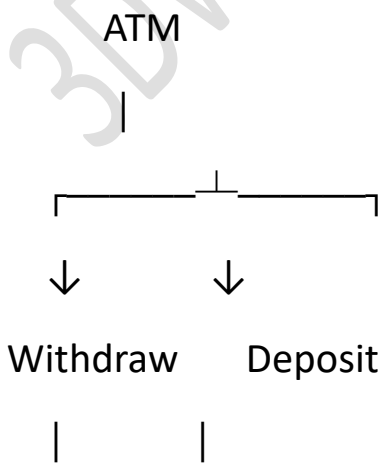
When you use an ATM, you select:

Withdraw Money

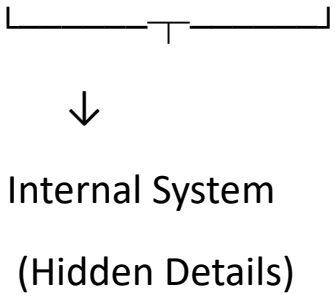
You don't need to know:

- How the bank server processes the request
- How the transaction is recorded
- How the cash dispenser works internally

You only interact with the necessary interface.



3DWEBSOFT FOUNDATION



12. Encapsulation vs Abstraction

Encapsulation	Abstraction
Protects/controls data and implementation details	Hides unnecessary implementation details
Focuses on access	Focuses on interface
Uses classes, methods, properties, naming conventions	Often uses abstract classes and methods
Example: private balance	Example: withdraw() without exposing banking internals

Easy way to remember

ENCAPSULATION

↓

"Protect the data"

ABSTRACTION

↓

"Hide the complexity"

3DWEBSOFT FOUNDATION

13. How Python Provides Abstraction

Python provides abstract base classes through the abc module.

We can use:

from abc import ABC, abstractmethod

An abstract class can define methods that child classes are required to implement.

14. Abstract Class

Example:

```
from abc import ABC, abstractmethod

class Employee(ABC):
    @abstractmethod
    def work(self):
        pass
```

Here:

Employee

is an abstract class.

It says:

Every employee should provide a work() method.

15. Implementing an Abstract Class

```
from abc import ABC, abstractmethod
```

3DWEBSOFT FOUNDATION

```
class Employee(ABC):
    @abstractmethod
    def work(self):
        pass
class Developer(Employee):
    def work(self):
        print("Developer writes software")
class Manager(Employee):
    def work(self):
        print("Manager manages the team")
d1 = Developer()
m1 = Manager()
d1.work()
m1.work()
```

Output

```
Developer writes software
Manager manages the team
```

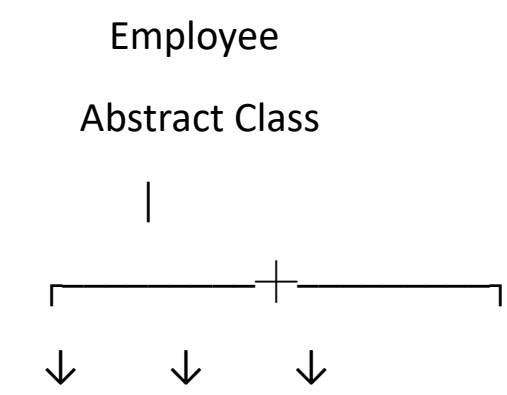
The parent class defines the required behavior, while child classes provide the implementation.

16. Why Use Abstract Classes?

Abstract classes are useful when different classes should follow the same structure.

SDWEBSOFT FOUNDATION

For example:



Every employee must implement:

work()

but each role can implement it differently.

17. Abstraction with a Shape Example

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

SUWEBSOFT FOUNDATION

```
    def area(self):
        return self.length * self.width
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius * self.radius
r = Rectangle(10, 5)
c = Circle(7)
print("Rectangle Area:", r.area())
print("Circle Area:", c.area())
```

Output

Rectangle Area: 50

Circle Area: 153.86

The user of the class simply calls:

area()

without needing to manage the calculation details themselves.

3DWEBSOFT FOUNDATION

18. Secure Class Design

A **secure class** should:

1. Protect sensitive data.
2. Validate input.
3. Provide controlled access.

4. Avoid exposing unnecessary implementation details.
5. Keep responsibilities organized.
6. Prevent invalid states where possible.

Example

Instead of:

account.balance = -10000

use:

account.deposit(5000)

account.withdraw(1000)

The class controls what is allowed.

19. Practical Project – Secure Bank Account

Let's create a secure Bank Account using **encapsulation**.

```
class BankAccount:

    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.__balance = balance
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Amount deposited successfully")
        else:
```

3JWEBSOFT FOUNDATION

```
            print("Invalid deposit amount")
    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount")
        elif amount > self.__balance:
            print("Insufficient balance")
        else:
            self.__balance -= amount
            print("Withdrawal successful")
    def get_balance(self):
        return self.__balance
    def display(self):
        print("\nAccount Holder:", self.account_holder)
        print("Balance:", self.__balance)
account = BankAccount("Rahul", 10000)
account.display()
account.deposit(5000)
account.withdraw(3000)
account.display()
```

Output

```
Account Holder: Rahul
Balance: 10000
Amount deposited successfully
```

3DWEBSOFT FOUNDATION

Withdrawal successful

Account Holder: Rahul

Balance: 12000

20. Why Is This Class More Secure?

The balance is:

self.__balance

rather than:

self.balance

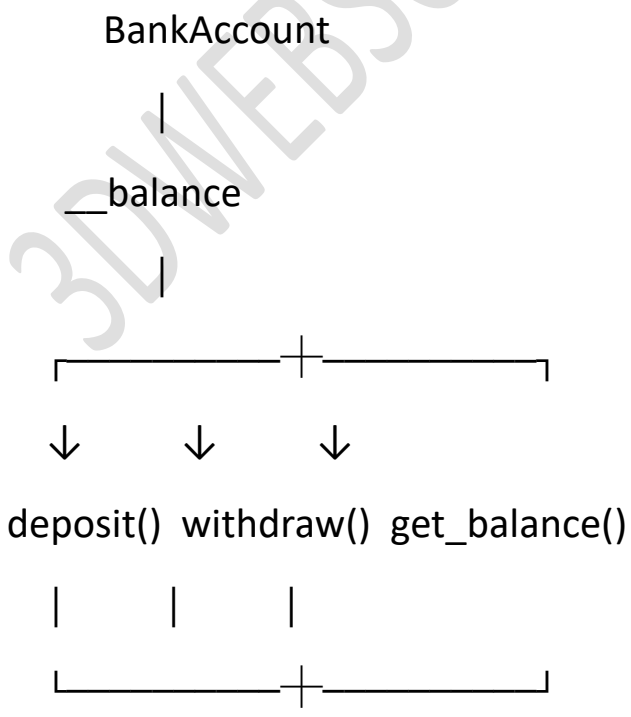
Users interact through:

deposit()

withdraw()

get_balance()

The class validates operations.



3DWEBSOFT FOUNDATION

↓

Controlled Access

21. Practical Project – Secure Student Class

Now let's apply encapsulation to our previous Student class.

```
class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.__marks = marks
    def get_marks(self):
        return self.__marks
    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks")
    def result(self):
        if self.__marks >= 35:
            return "PASS"
        else:
            return "FAIL"
```

3DWEBSOFT FOUNDATION

```
    def display(self):
        print("Name:", self.name)
        print("Roll No:", self.roll_no)
        print("Marks:", self.__marks)
        print("Result:", self.result())
student = Student("Asha", 101, 85)
student.display()
student.set_marks(95)
print("\nAfter Updating Marks:")
student.display()
```

Output

```
Name: Asha
Roll No: 101
Marks: 85
Result: PASS

After Updating Marks:
Name: Asha
Roll No: 101
Marks: 95
Result: PASS
```

22. Python Property

3DWEBSOFT FOUNDATION

Python provides @property to create a clean interface for controlled access.

Example:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.__marks = marks
    @property
    def marks(self):
        return self.__marks
    @marks.setter
    def marks(self, value):
        if 0 <= value <= 100:
            self.__marks = value
        else:
            print("Invalid marks")
s1 = Student("Rahul", 80)
print(s1.marks)
s1.marks = 90
print(s1.marks)
```

Output

```
80
90
```

3DWEBSOFT FOUNDATION

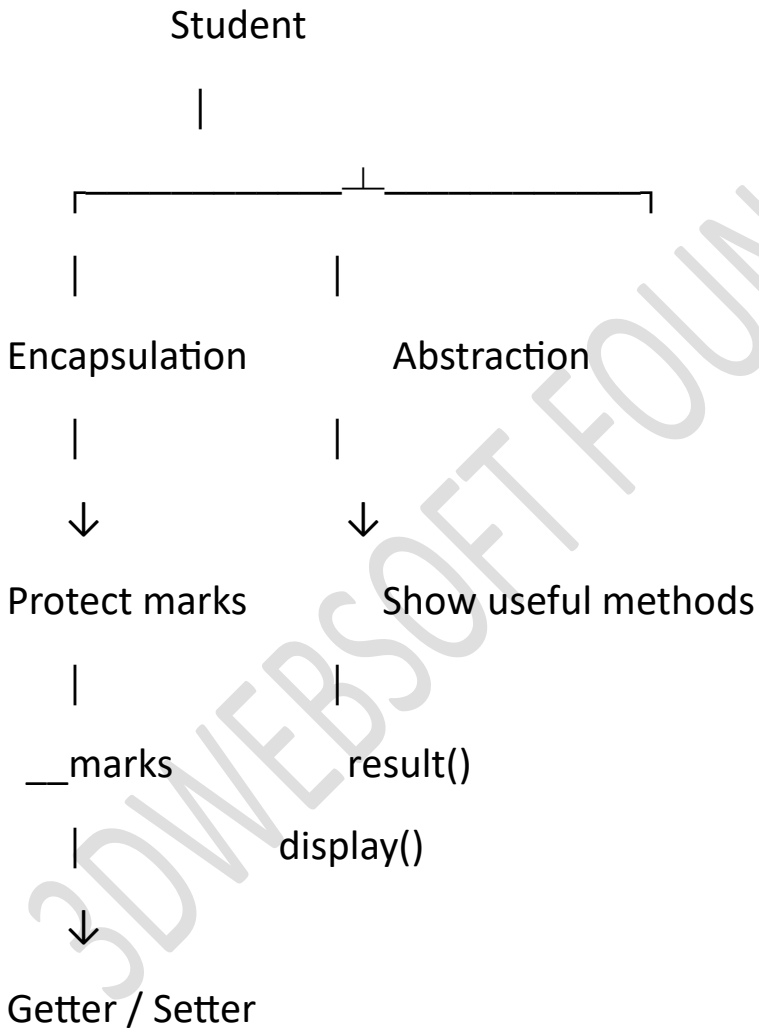
The syntax looks like normal attribute access:

s1.marks

but the class controls how the value is accessed and changed.

23. Encapsulation + Abstraction Together

A good object-oriented design can use both.



24. Secure Employee Class

Let's apply the same idea to Employee Management.

class Employee:

3DWEBSOFT FOUNDATION

```
    def __init__(self, name, emp_id, salary):
        self.name = name
        self.emp_id = emp_id
        self.__salary = salary

    def get_salary(self):
        return self.__salary

    def set_salary(self, salary):
        if salary >= 0:
            self.__salary = salary
        else:
            print("Salary cannot be negative")
    def display(self):
        print("Name:", self.name)
        print("Employee ID:", self.emp_id)
        print("Salary:", self.__salary)
e1 = Employee("Rahul", 101, 50000)
e1.display()
e1.set_salary(55000)
print("\nUpdated Employee:")
e1.display()
```

Output

3UWEBSOFT FOUNDATION

Name: Rahul

Employee ID: 101

Salary: 50000

Updated Employee:

Name: Rahul

Employee ID: 101

Salary: 55000

25. Secure Class Design – Best Practices

1. Protect sensitive data

Use private-style attributes where appropriate:

self.__balance

2. Validate input

if amount > 0:

3. Don't expose unnecessary details

Provide simple methods:

withdraw()

instead of exposing internal transaction logic.

4. Use meaningful method names

Good:

calculate_salary()

Less clear:

do_it()

3DWEBSOFT FOUNDATION

5. Keep each class focused

A Student class should mainly manage student-related behavior.

26. Common Mistakes

Mistake 1 – Incorrect private attribute

❌

self._balance

This is protected-style, not private-style.

✅

self.__balance

Mistake 2 – Forgetting self

❌

def get_balance():

✅

def get_balance(self):

Mistake 3 – Forgetting @abstractmethod

For an abstract method:

from abc import ABC, abstractmethod

then:

@abstractmethod

3DWEBSOFT FOUNDATION

```
def work(self):
    pass
```

Mistake 4 – Trying to create an abstract class object

If a class contains an unimplemented abstract method:

```
class Employee(ABC):
    @abstractmethod
    def work(self):
        pass
```

we cannot normally create:

```
Employee()
```

A concrete child class must implement the abstract method.

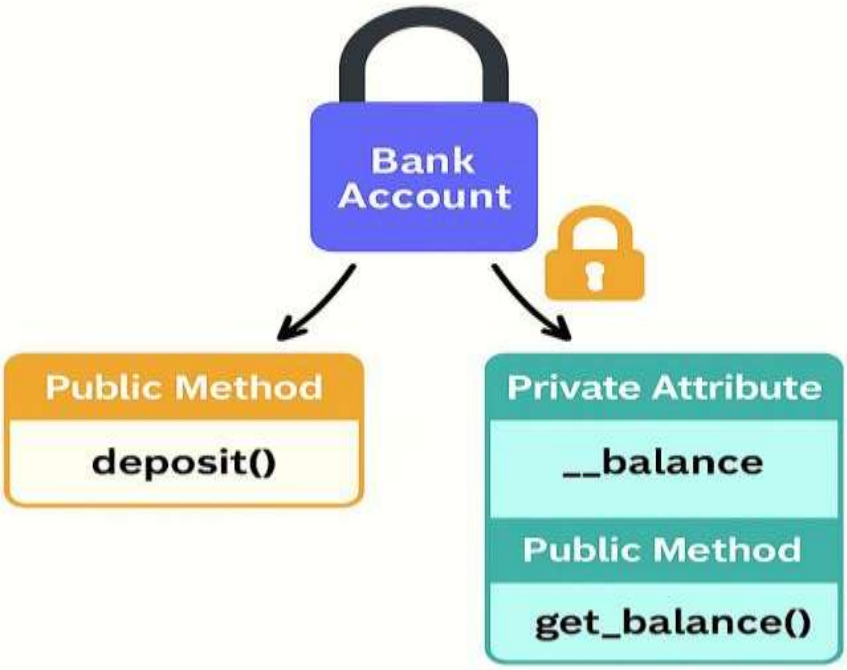
27. Encapsulation and Abstraction in Real Life



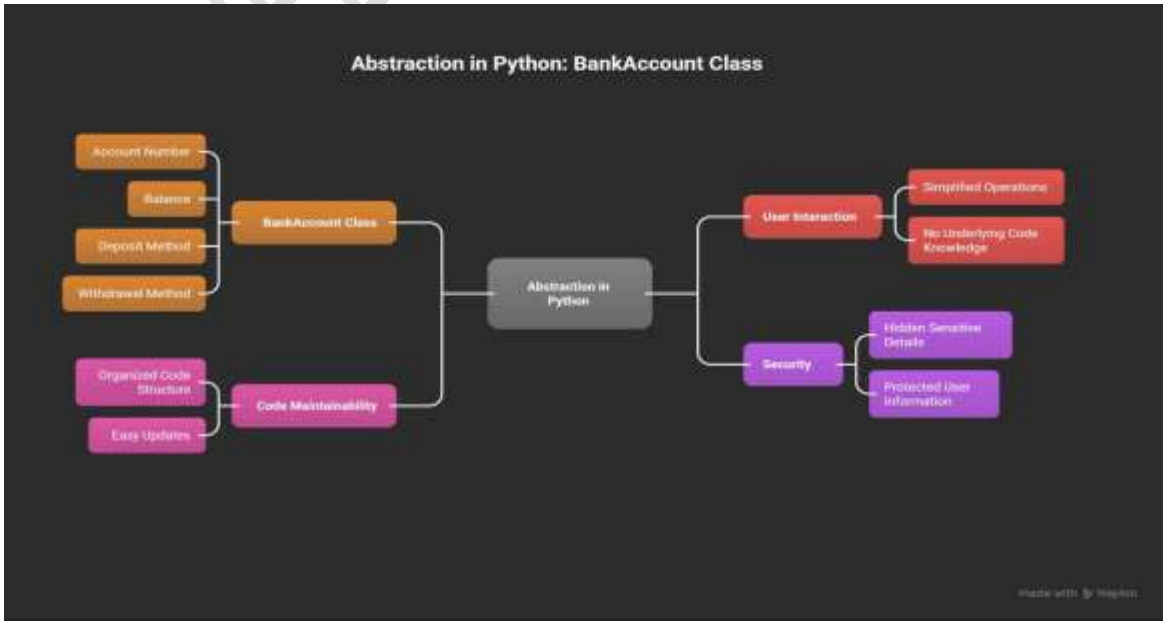
SOFT FOUNDATION

Encapsulation in Python

Hide internal details, expose only what's necessary



@abdul-quadir Follow for more Python tips!



Real World	Encapsulation	Abstraction
ATM	Protects account information	Shows simple ATM buttons
Bank	Protects balance/transactions	Provides deposit/withdraw
Car	Protects internal components	Driver uses steering/brake
School	Protects student records	Shows useful student operations
E-commerce	Protects order data	Shows simple checkout process

28. Classroom Activity – Student Security

Create a class:

class Student:

with:

name

roll_no

__marks

Create methods:

get_marks()

set_marks()

result()

3DWEBSOFT FOUNDATION

display()

Rules

- Marks must be between 0 and 100.
- Negative marks should not be accepted.
- Marks above 100 should not be accepted.

29. Classroom Activity – Bank Account

Create:

class BankAccount:

Attributes:

account_holder

__balance

Methods:

deposit()

withdraw()

get_balance()

Rules:

- Deposit must be greater than 0.
- Withdrawal must be greater than 0.
- Withdrawal cannot exceed balance.

30. Mini Project – Secure Employee Management

Requirements

3DWEBSOFT FOUNDATION

Create an Employee class with:

name

employee_id

__salary

Methods:

get_salary()

set_salary()

calculate_bonus()

display()

Rules

- Salary cannot be negative.
- Bonus should be calculated using a method.
- Salary should not be modified directly.
- Display employee details using a method.

Challenge

Create:

Developer

Manager

Designer

as child classes and use an abstract work() method.

31. Complete Advanced Practical

This program combines:

3DWEBSOFT FOUNDATION

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

```
from abc import ABC, abstractmethod
class Employee(ABC):
    def __init__(self, name, emp_id, salary):
        self.name = name
        self.emp_id = emp_id
        self.__salary = salary
    def get_salary(self):
        return self.__salary
    def set_salary(self, salary):
        if salary >= 0:
            self.__salary = salary
        else:
            print("Invalid salary")
    def display(self):
        print("\nEmployee ID:", self.emp_id)
        print("Name:", self.name)
        print("Salary:", self.__salary)
    @abstractmethod
    def work(self):
```

3DWEBSOFT FOUNDATION

```
        pass
class Developer(Employee):
    def work(self):
        print(self.name, "is developing software")
class Manager(Employee):
    def work(self):
        print(self.name, "is managing the team")
class Designer(Employee):
    def work(self):
        print(self.name, "is designing graphics")
employees = [
    Developer("Rahul", 101, 50000),
    Manager("Asha", 102, 70000),
    Designer("Ravi", 103, 45000)
]
for employee in employees:
    employee.display()
    employee.work()
```

Output

```
Employee ID: 101
Name: Rahul
Salary: 50000
Rahul is developing software
```

3JWEBSOFT FOUNDATION

Employee ID: 102

Name: Asha

Salary: 70000

Asha is managing the team

Employee ID: 103

Name: Ravi

Salary: 45000

Ravi is designing graphics

Concepts used

Employee

```
  |
  ├── Encapsulation
  |     └── __salary
  |
  ├── Abstraction
  |     └── abstract work()
  |
  ├── Inheritance
  |     └── Developer / Manager / Designer
  |
  └── Polymorphism
        └── different work()
```

3DWEBSOFT FOUNDATION

32. Quick Revision

Encapsulation

Protect and control access to data.

Example:

self.__balance

Abstraction

Hide unnecessary implementation details and expose essential functionality.

Example:

withdraw()

Getter

Reads data:

get_balance()

Setter

Updates data with control:

set_balance()

Abstract Class

A class designed to define required behavior for subclasses.

class Employee(ABC):

Abstract Method

A method that subclasses are expected to implement.

@abstractmethod

def work(self):

3DWEBSOFT FOUNDATION

```
    pass
```

33. Important Questions

Q1. What is encapsulation?

Encapsulation is the process of combining data and methods in a class and controlling access to the data.

Q2. What does __ before an attribute indicate?

It creates a private-style attribute using Python's name-mangling mechanism.

Q3. What is abstraction?

Abstraction hides unnecessary implementation details and exposes essential functionality.

Q4. What module provides abstract base classes?

abc

Q5. What is @abstractmethod?

It marks a method that concrete subclasses are expected to implement.

Q6. What is a getter?

A method/property used to access a value.

Q7. What is a setter?

A method/property used to update a value, usually with validation.

34. MCQs

JOWEBSOFT FOUNDATION

1. Which concept protects and controls access to data?

A. Inheritance
B. Encapsulation
C. Looping
D. Compilation

Answer: B

2. Which symbol is commonly used for a private-style attribute?

A. #
B. _
C. __
D. @

Answer: C

3. Which module provides ABC?

A. math
B. random
C. abc
D. os

Answer: C

4. Which decorator creates an abstract method?

A. @method
B. @abstractmethod
C. @private
D. @class

Answer: B

5. Abstraction mainly focuses on:

3DWEBSOFT FOUNDATION

A. Hiding unnecessary implementation details
B. Creating loops
C. Creating files
D. Printing values

Answer: A

6. Which is an example of encapsulation?

A. self.__balance
B. for loop
C. if statement
D. import math

Answer: A

35. Practical Assignment

Secure Student Management System

Create a Student class with:

Private Data

__marks

Public Data

name

roll_no

Methods

get_marks()

set_marks()

calculate_grade()

3DWEBSOFT FOUNDATION

display()

Validation

0 ≤ marks ≤ 100

Extension Challenge

Create an abstract class:

class Person(ABC):

with:

@abstractmethod

def display_role(self):

    pass

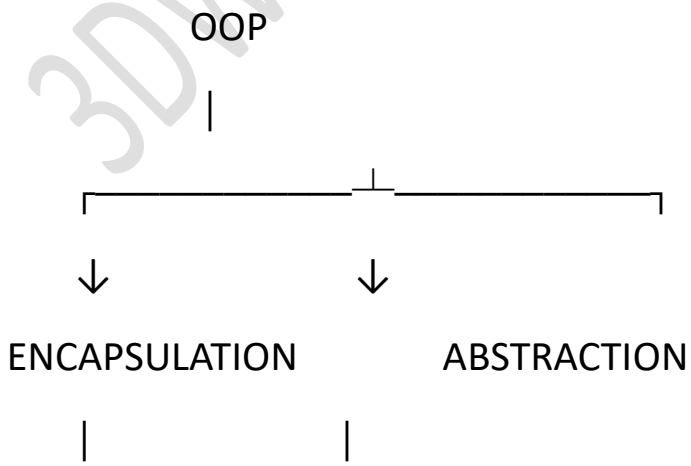
Then create:

Student

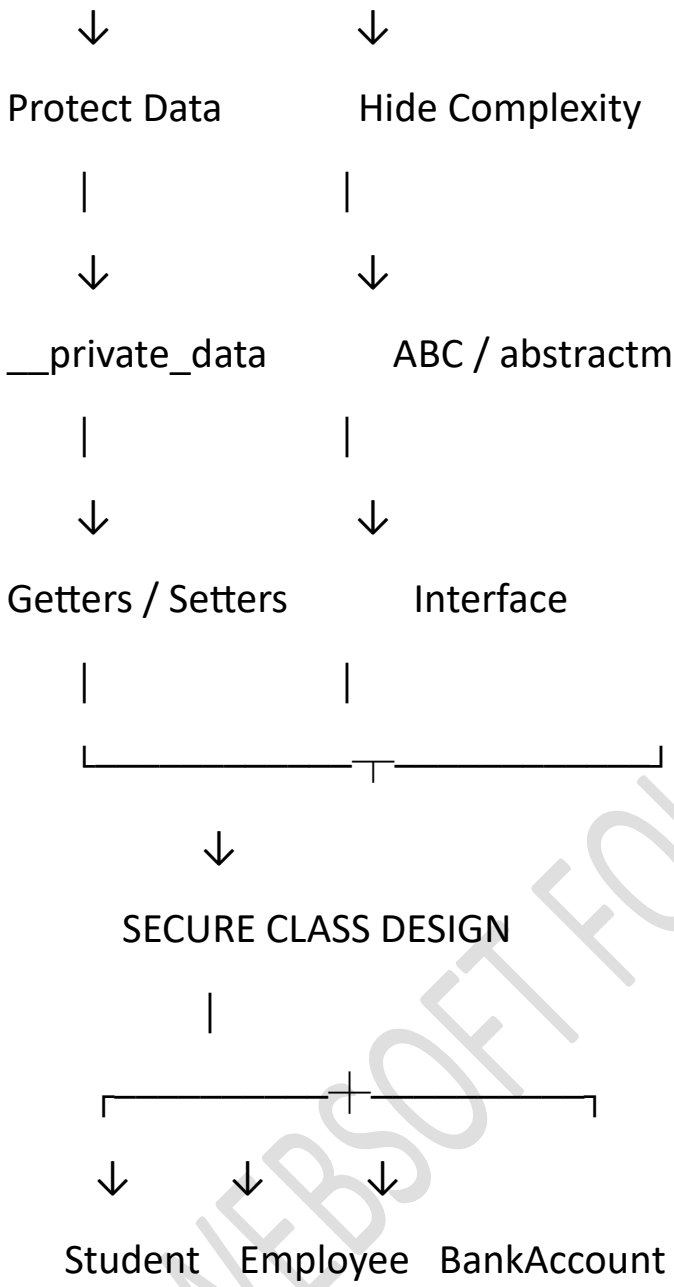
Teacher

as child classes.

36. Concept Map



3DWEBSOFT FOUNDATION



2DWEBSOFT FOUNDATION

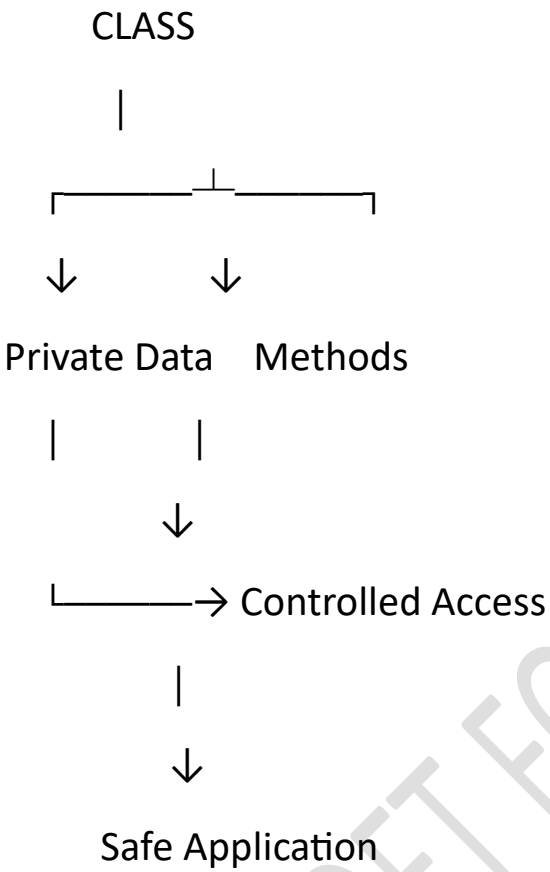
Final Takeaway

Remember the four major OOP concepts learned so far:

1. Encapsulation → Protect the data
2. Abstraction → Hide complexity
3. Inheritance → Reuse code

4. Polymorphism → Same interface, different behavior

For **secure class design**, the basic idea is:



3DWEBSOFT FOUNDATION